

C2SafeRust-LLM

Module 7: GUI & Report Assembler -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 7: GUI & Report Assembler

Line-by-Line Code Explanation for Beginners

Files: `src/module7_gui/mod.rs`, `report_assembler.rs`, `gui_app.rs`

What does Module 7 do? Module 7 is the *FACE* of the application — the part the user actually sees and interacts with. It has *TWO* jobs: 1. **REPORT ASSEMBLER** (`report_assembler.rs`): Collect all outputs from Modules 1-6, check which of the 6 C Idioms were found, and build a formatted markdown report. 2. **GUI APPLICATION** (`gui_app.rs`): Render an interactive desktop window using the `egui` + `eframe` framework. The GUI shows: - The original C code (left panel) - The generated Safe Rust code (right panel) - A KPI metric bar (lines, functions, call graph, repair iterations) - A tabbed bottom drawer with: 6 Idioms Table | Executive Report | Repair History - A status banner with real-time feedback (loading, success, error)

```
All Module Outputs  ██████████ report_assembler.rs
                        |
                        v
                    TranslationReport (struct)
                        |
                        v
                    gui_app.rs
                        |
                        v
                [Desktop Window rendered by egui/eframe]
```

File 1: `src/module7_gui/mod.rs`

(The Launch Entry Point)

```

pub mod report_assembler;
pub mod gui_app;

pub use report_assembler::{assemble_translation_report, TranslationReport, IdiomStatus};
pub use gui_app::C2SafeRustApp;

/// Primary entry point for Module 7: Launches the egui GUI.
pub fn launch_gui() -> Result<(), eframe::Error> {
    let options = eframe::NativeOptions {
        viewport: eframe::egui::ViewportBuilder::default()
            .with_inner_size([1440.0, 900.0])
            .with_min_inner_size([1100.0, 700.0])
            .with_title("C2SafeRust-LLM Transpilation Framework"),
        ..Default::default()
    };

    eframe::run_native(
        "C2SafeRust-LLM Transpilation Framework",
        options,
        Box::new(|cc| Ok(Box::new(C2SafeRustApp::new(cc)))),
    )
}

```

	Explanation		
	Configuration for the native desktop window. A struct with many fields.		
	Builder pattern for window properties. `default()` sets sensible starting values.		
	Initial window size: 1440 wide, 900 tall, in logical pixels. Array `[f32; 2]` — two floats.		
	Minimum size the user can shrink the window to.		
	The text shown in the OS title bar.		
	"Fill all other fields of `NativeOptions` with their default values." The `..` is Rust's struct update syntax — copies all remaining fields from the given instance.		
	Launch the actual window. Blocks until the window is closed. Returns `Result<(), eframe::Error>`.		
	cc\	Ok(Box::new(C2SafeRustApp::new(cc)))`	The app fact access to the heap-allocated `Result`.

What is eframe? **eframe** is a Rust GUI framework built on top of **egui** (Emoji GUI). - **egui** = the UI LIBRARY (widgets, layouts, rendering) - **eframe** = the APPLICATION FRAMEWORK (window creation, event loop, OpenGL backend) The model is "retained mode GUI" — the **update()** function is called EVERY FRAME (60+ times per second) and redraws the entire UI from scratch. There's no separate "update state" step — all layout and rendering happen together.

File 2: `src/module7_gui/report_assembler.rs`

(Collecting Outputs into a `TranslationReport`)

Lines 8-14: The `IdiomStatus` Struct

```
#[derive(Debug, Clone)]
pub struct IdiomStatus {
    pub idiom_number: usize,
    pub name: &'static str,
    pub detected: bool,
    pub resolution: String,
}
```

Field	Type	Example
<code>`idiom_number`</code>	<code>`usize`</code>	<code>`1`, `2`, ..., `6`</code>
<code>`name`</code>	<code>`&'static str`</code>	<code>"Aliased &mut; parameters"</code>
<code>`detected`</code>	<code>`bool`</code>	<code>`true`</code> (if this idiom was found in the C code)
<code>`resolution`</code>	<code>`String`</code>	Description of how it was handled

What is `&'static str`?

```
&str          = a string slice – a reference to some string data
'static        = a LIFETIME annotation – lives for the ENTIRE program duration
&'static str = a string slice that is embedded in the binary at compile time
```

This is the type of string LITERALS like `"Aliased &mut; parameters"`.

They're stored in the program's read-only data segment — never freed.

Using `&'static str` instead of `String` is more memory-efficient for fixed text that never changes at runtime.

Lines 16-29: The `TranslationReport` Struct

```
#[derive(Debug, Clone)]
pub struct TranslationReport {
    pub source_file_name: String,
    pub c_lines: usize,
    pub c_bytes: usize,
    pub functions_count: usize,
    pub call_graph_nodes: usize,
    pub call_graph_edges: usize,
    pub idioms_summary: Vec<IdiomStatus>,
    pub repair_iterations: usize,
    pub compilation_successful: bool,
    pub final_rust_code: String,
    pub formatted_markdown_report: String,
}
```

This is the "summary of summaries" — everything the user would want to know after a transpilation run, bundled into one struct. Think of it as the data that feeds both the GUI's bottom panel and any exported report file.

Field	Data source
`source_file_name`	From `validated.file_path` (Module 1)
`c_lines`, `c_bytes`	From `validated.line_count`, `.byte_size` (Module 1)
`functions_count`	From `tu_ast.functions.len()` (Module 2)
`call_graph_nodes/edges`	From `summaries.call_graph.node_count()` (Module 4)
`idioms_summary`	Built locally — 6 checks against `summaries` (Module 4 D2)
`repair_iterations`	From `repair_res.iterations_used` (Module 6)
`compilation_successful`	From `repair_res.final_success` (Module 6)
`final_rust_code`	From `repair_res.final_rust_code` (Module 6)
`formatted_markdown_report`	Built locally — markdown string

Lines 31-156: The `assemble_translation_report()` Function

```
pub fn assemble_translation_report(
    validated: &ValidatedCSource,
    tu_ast: &TranslationUnitAST,
    _semantic_models: &HashMap<String, SemanticModel>,
    summaries: &InterproceduralSummaries,
    repair_res: &RepairResult,
) -> TranslationReport {
```

Note `_semantic_models`: the leading `_` tells Rust (and readers) this parameter is intentionally unused in this function — it's passed for API consistency. Without the `_`, Rust would emit an "unused variable" warning.

Idiom 1 Detection (Lines 42-53):

```
let idiom1_detected = summaries.alias_verdicts
    .values()
    .any(|verdicts| verdicts.iter().any(|v| v.aliases));
idioms.push(IdiomStatus {
    idiom_number: 1,
    name: "Aliased &mut parameters",
    detected: idiom1_detected,
    resolution: if idiom1_detected {
        "Interprocedural points-to alias analysis over call-site arguments".to_string()
    } else {
        "No call-site aliasing detected".to_string()
    },
});
```

ashMap			
	verdicts.iter().any(	v\	v.aliases))`

Idiom 2 Detection (Lines 55-66):

```
let idiom2_detected = summaries.ownership_transfer_summaries
    .values()
    .any(|s| matches!(
        s.transfer_type,
        crate::data_stores::interprocedural_summaries::OwnershipTransferType::ConsumesOwnership
    ));
```

- `.values()` — iterate over all `OwnershipTransferSummary` structs.
- `matches!(s.transfer_type, OwnershipTransferType::ConsumesOwnership)` — returns `true` if the transfer type is `ConsumesOwnership`.
- Note the full path `crate::data_stores::...::OwnershipTransferType::ConsumesOwnership` — when there's no `use` import, the full crate path is used inline.

Idiom 3 Detection (Lines 68-79): Identical pattern but for `CreatesOwnership`.

Idiom 4 Detection (Lines 81-92):

```
let idiom4_detected = summaries.alias_verdicts
    .values()
    .any(|verdicts| verdicts.iter()
        .any(|v| v.arg1.contains('+') || v.arg2.contains('+')));
```

If any argument in any alias verdict contains a `+` character, it's pointer arithmetic — Idiom 4.

Idiom 5 Detection (Lines 94-105):

```
let idiom5_detected = !summaries.cross_function_union_resolutions.is_empty();
```

Simple! If the HashMap has ANY cross-function union resolutions at all, Idiom 5 was detected.

`.is_empty()` returns `true` if the HashMap has zero entries; `!` negates it.

Idiom 6 Detection (Lines 107-118):

```
let idiom6_detected = !summaries.data_dependent_ownership_flags.is_empty();
```

Same pattern — if the Vec of `DataDepFlags` has any entries, Idiom 6 was detected.

Building the Markdown Report (Lines 120-141)

```
let mut markdown = String::new();
markdown.push_str(&format!(
    "# Translation Report for {:?}\n\n",
    validated.file_path.file_name().unwrap_or_default()
));
markdown.push_str(&format!(
    "- **Input C File**: {} bytes, {} lines\n",
    validated.byte_size, validated.line_count
));
// ... (more push_str calls)
markdown.push_str("## 6 C Idioms Taxonomy Summary\n\n");
for idm in &idioms {
    let status_str = if idm.detected { "DETECTED" } else { "NOT DETECTED" };
    markdown.push_str(&format!(
        "- **Idiom {} ({}):** [{}] - {}\n",
        idm.idiom_number, idm.name, status_str, idm.resolution
    ));
}

markdown.push_str("\n## Repair History\n\n");
if repair_res.repair_logs.is_empty() {
    markdown.push_str("No repair iterations required (clean translation).\n");
} else {
    for log in &repair_res.repair_logs {
        markdown.push_str(&format!(
            "- Iteration {}: Source: {} | Note: {}\n",
            log.iteration, log.patch_source, log.explanation
        ));
    }
}
```

Code	Explanation
<code>\validated.file_path.file_name()</code>	<code>\file_path</code> is a <code>\PathBuf</code> . <code>\file_name()</code> extracts the filename component (e.g., <code>"test.c"</code>). Returns <code>\Option<&OsStr></code> .
<code>\unwrap_or_default()</code>	If <code>\file_name()</code> returns <code>\None</code> (e.g., path is <code>\</code>), use the default <code>\OsStr</code> (empty).

Code	Explanation
<code>`{:?}` in format</code>	Debug format — prints <code>`OsStr`</code> values like <code>``"test.c"``</code> (with quotes).
<code>`&format;!(...)`</code>	<code>`format!`</code> returns a <code>`String`</code> . <code>`&`</code> borrows it as <code>`&str;`</code> for <code>`push_str`</code> .
<code>`summaries.call_graph.node_count()`</code>	petgraph built-in method.
<code>`if repair_res.repair_logs.is_empty()`</code>	Show a different message depending on whether repairs were needed.

The Unit Test (Lines 158-185)

```
#[test]
fn test_report_assembly() {
    let val = ValidatedCSource {
        file_path: PathBuf::from("test.c"),
        raw_content: "int main() {}".to_string(),
        line_count: 1,
        byte_size: 13,
    };

    let tu_ast = TranslationUnitAST::new(); // Empty AST
    let models = HashMap::new();           // No semantic models
    let summaries = InterproceduralSummaries::new(); // No D2 data
    let repair_res = RepairResult {
        final_success: true,
        iterations_used: 1,
        repair_logs: vec![],
        final_rust_code: "pub fn main() {}".to_string(),
    };

    let report = assemble_translation_report(&val, &tu_ast, &models, &summaries, &repair_res);
    assert_eq!(report.idioms_summary.len(), 6); // All 6 idioms checked
    assert!(report.compilation_successful);     // success flag propagated
}
```

File 3: src/module7_gui/gui_app.rs

(The Full Interactive Desktop GUI)

egui Concepts Before We Dive In: - `egui::Ui` — the main drawing context. You call methods on it to add widgets. - `Frame::none().fill(...).show(ui, |ui| { ... })` — draw a styled rectangle, run code inside it. - `egui::TopBottomPanel::top("id")` — a panel anchored to the top of the window. - `egui::SidePanel::left("id")` — a panel anchored to the left, resizable. - `egui::CentralPanel::default()` — the remaining middle area after panels claim space. - Everything uses CLOSURES: `|ui| { ... }` — code that runs inside the panel's draw context. - `RichText::new("text").strong().size(14.0).color(Color32::WHITE)` — styled text. - `Color32::from_rgb(r, g, b)` — a color from 0-255 RGB values.

Lines 6-22: The PipelineResult Transfer Struct

```
pub struct PipelineResult {
    pub file_path: String,
    pub original_c_code: String,
    pub generated_rust_code: String,
    pub markdown_report_text: String,
    pub c_lines: usize,
    pub c_bytes: usize,
    pub func_count: usize,
    pub call_nodes: usize,
    pub call_edges: usize,
    pub repair_iterations: usize,
    pub idioms_status_display: Vec<(usize, String, bool, String)>,
    pub repair_logs: Vec<(usize, String, String)>,
    pub is_success: bool,
    pub is_error: bool,
    pub status_message: String,
}
```

This struct is the "package" sent from the background worker thread to the GUI thread. It contains ALL data the GUI needs to update itself after a transpilation run. Note `idioms_status_display: Vec<(usize, String, bool, String)>`: A Vec of TUPLES — each (id, name, detected, resolution). Tuples group multiple values without naming them as a struct. Used here for compact data transfer.

Lines 24-44: The App State C2SafeRustApp

```
#[derive(Default)]
pub struct C2SafeRustApp {
    pub file_path_input: String,          // The text box input
    pub original_c_code: String,          // Left panel content
    pub generated_rust_code: String,      // Right panel content
    pub status_message: String,          // Status banner text
    pub is_success: bool,                 // Green banner
    pub is_error: bool,                   // Red banner
    pub is_transpiling: bool,             // Spinner + disabled button
    pub active_tab: usize,                // 0, 1, or 2 (bottom drawer tab)
    pub markdown_report_text: String,
    pub idioms_status_display: Vec<(usize, String, bool, String)>,
    // ... metrics ...
    pub transpilation_rx: Option<Receiver<PipelineResult>>,
}
```

Important Field	Type	Explanation
<code>#[derive(Default)]</code>	Macro	Auto-generates a <code>Default</code> implementation: all Strings → <code>""</code> , all bools → <code>false</code> , all <code>usize</code> → <code>0</code> , all <code>Option</code> → <code>None</code> , all <code>Vec</code> → empty.

Important Field	Type	Explanation
<code>`transpilation_rx: Option>`</code>	The channel receiver	<code>`None`</code> when idle, <code>`Some(rx)`</code> while a background transpilation is running. The GUI polls <code>`rx.try_recv()`</code> every frame.
<code>`Receiver`</code>	From <code>`std::sync::mpsc`</code>	The receiving end of a Multi-Producer Single-Consumer (mpsc) channel. Used for thread-safe communication from background thread → GUI thread.

Lines 46-62: `new()` — Dark Theme Initialization

```
pub fn new(cc: &eframe::CreationContext<'_>) -> Self {
    let mut visuals = egui::Visuals::dark();
    visuals.panel_fill = Color32::from_rgb(18, 20, 29); // Very dark navy
    visuals.window_fill = Color32::from_rgb(24, 27, 38); // Dark navy
    visuals.widgets.noninteractive.bg_fill = Color32::from_rgb(28, 32, 46);
    visuals.widgets.inactive.bg_fill = Color32::from_rgb(38, 44, 64);
    visuals.widgets.hovered.bg_fill = Color32::from_rgb(58, 66, 94);
    visuals.widgets.active.bg_fill = Color32::from_rgb(99, 102, 241); // Indigo
    cc.egui_ctx.set_visuals(visuals);

    let mut app = Self::default();
    app.file_path_input = "benchmarks/sample_test.c".to_string();
    app.load_file_from_path();
    app
}
```

Code	Explanation
<code>`egui::Visuals::dark()`</code>	Start from egui's built-in dark theme, then customize.
<code>`Color32::from_rgb(18, 20, 29)`</code>	Dark navy background (#12141D in hex).
<code>`Color32::from_rgb(99, 102, 241)`</code>	Indigo accent (#6366F1) — used for active buttons.
<code>`cc.egui_ctx.set_visuals(visuals)`</code>	Apply our custom theme to the entire application context.
<code>`Self::default()`</code>	Create the app with all fields at their default values (from <code>`#[derive(Default)]`</code>).
<code>`app.load_file_from_path()`</code>	Pre-load the sample C file into the code preview on startup.

Lines 64-93: `load_file_from_path()` — File Loading

```

pub fn load_file_from_path(&mut self) {
    let path = Path::new(&self.file_path_input);
    if path.exists() && path.is_file() {
        if let Ok(content) = std::fs::read_to_string(path) {
            self.original_c_code = content.clone();
            self.c_lines = content.lines().count();
            self.c_bytes = content.len();
            self.generated_rust_code = "// Click '■ Run Transpilation'...".to_string();
            self.status_message = format!(
                "■ Loaded '{}' ({} lines, {} bytes).",
                self.file_path_input, self.c_lines, self.c_bytes
            );
            return;
        }
    }
    // File not found:
    self.original_c_code = format!("// Error: File not found at '{}'", self.file_path_input);
    self.is_error = true;
    self.status_message = format!("■■ File not found: '{}'", self.file_path_input);
}

```

Code	Explanation
<code>&mut; self</code>	Takes a mutable reference to self — can modify all fields.
<code>path.exists() && path.is_file()</code>	Guard against directories and non-existent paths. Both conditions must be true.
<code>content.lines().count()</code>	Count newlines to get line count. <code>.lines()</code> returns an iterator; <code>.count()</code> consumes it and returns the total.
<code>content.len()</code>	Number of BYTES (not characters). For ASCII C code, this equals character count.
<code>return</code>	Early return after successful load — skip the error handling below.

Lines 95-105: `open_native_file_dialog()` — OS File Picker

```

pub fn open_native_file_dialog(&mut self) {
    if let Some(file_path) = FileDialog::new()
        .set_title("Select C Source File for Transpilation")
        .pick_file()
    {
        self.file_path_input = file_path.to_string_lossy().to_string();
        self.load_file_from_path();
    }
}

```

Code	Explanation
<code>FileDialog::new()</code>	From the <code>`rfd`</code> crate (Rust File Dialog). Creates a native OS open-file dialog.

Code	Explanation
<code>`.set_title("Select C Source File...")`</code>	Sets the dialog window title.
<code>`.pick_file()`</code>	Opens the dialog synchronously. Returns <code>`Option`</code> — <code>`None`</code> if user cancels, <code>`Some(path)`</code> if a file is selected.
<code>`if let Some(file_path) = ...`</code>	Only proceed if user picked a file (not cancelled).
<code>`file_path.to_string_lossy().to_string()`</code>	Convert <code>`PathBuf`</code> → <code>`String`</code> . <code>`.to_string_lossy()`</code> handles OS-specific path encoding.

Lines 107-132: `start_transpilation()` — *Background Thread Launch*

```
pub fn start_transpilation(&mut self) {
    if self.is_transpiling { return; } // Prevent double-click

    let path = Path::new(&self.file_path_input);
    if !path.exists() || !path.is_file() {
        self.load_file_from_path(); // Show error
        return;
    }

    self.is_transpiling = true;
    self.is_success = false;
    self.is_error = false;
    self.status_message = format!("■ Transpiling '{}'" in background...", self.file_path_input);

    let (tx, rx) = channel();
    self.transpilation_rx = Some(rx);
    let path_str = self.file_path_input.clone();

    std::thread::spawn(move || {
        let result = execute_transpilation_pipeline(&path_str);
        let _ = tx.send(result);
    });
}
```

The Channel + Thread Pattern — Step by Step:

	Code	Explanation		
	<code>`let (tx, rx) = channel()`</code>	Create a one-shot mpsc channel. <code>`tx`</code> = transmitter (sender), <code>`rx`</code> = receiver.		
	<code>`self.transpilation_rx = Some(rx)`</code>	Store the RECEIVER in the app state. The GUI will poll this every frame.		
	<code>`let path_str = self.file_path_input.clone()`</code>	Clone the path string — the closure needs ownership of data it captures. We can't capture <code>`self`</code> (it's borrowed by <code>`&mut; self`</code>).		

	Code	Explanation		
	<code>`std::thread::spawn(move \</code>	<code>\</code>	<code>{ ... })`</code>	Start a NEW closure TAK and `path_st independent
	<code>`execute_transpilation_pipeline(&path;_str)`</code>	The full pipeline (Modules 1-7) runs in the background thread. This takes seconds.		
	<code>`tx.send(result)`</code>	When done, send the `PipelineResult` through the channel. The GUI thread can receive it.		
	<code>`let _ = tx.send(...)`</code>	Discard the `Result<(), SendError>` — if the GUI closed, the error is silently ignored.		

Why a background thread? The GUI runs on the MAIN THREAD and must repaint 60 times per second. If we ran the pipeline on the main thread, the window would FREEZE for several seconds while compiling. The background thread keeps the GUI responsive. `egui` does NOT provide `async`; we use `std::thread::spawn` for concurrency.

Lines 135-290: `execute_transpilation_pipeline()` — All 7 Stages

```

fn execute_transpilation_pipeline(path_str: &str) -> PipelineResult {
    let path = PathBuf::from(path_str);

    // Stage 1: Input Validation
    let validated = match crate::module1_validation::validate_input(&path) {
        Ok(src) => src,
        Err(err) => return PipelineResult { is_error: true, status_message: format!("■ Validation Error: {}", err),
        original_c_code: String::new(),
        // ... all other fields set to empty/false defaults ...
    };

    let original_c_code = validated.raw_content.clone();

    // Stage 2: Preprocessing & AST
    let module2_out = match crate::module2_ast::preprocess_and_parse(&validated) { ... };

    // Stage 3: Per-Function Semantic Analysis (never fails)
    let d1 = crate::module3_semantic::analyze_translation_unit_semantics(&module2_out.ast);

    // Stage 4: Interprocedural Analysis (never fails)
    let d2 = crate::module4_interprocedural::run_interprocedural_analysis(&module2_out.ast, &d1);

    // Stage 5: Ownership & Codegen
    let out_rs = Path::new("output.rs");
    let codegen_res = match crate::module5_codegen::generate_rust_code(&module2_out.ast, &d1, &out_rs) { ... };

    // Stage 6: Compiler Validation & LLM Repair (max 3 iterations)
    let repair_res = match crate::module6_repair::run_repair_loop(&validated.raw_content, &codegen_res) { ... };

    // Stage 7: Report Assembly
    let report = crate::module7_gui::report_assembler::assemble_translation_report(&validated, &repair_res);

    // Convert for GUI display
    let idioms_status_display = report.idioms_summary.iter()
        .map(|i| (i.idiom_number, i.name.to_string(), i.detected, i.resolution.clone()))
        .collect();

    let repair_logs = repair_res.repair_logs.iter()
        .map(|l| (l.iteration, l.patch_source.clone(), l.explanation.clone()))
        .collect();

    PipelineResult { ... }
}

```

The Error-Early-Return Pattern for each stage:

```

let validated = match crate::module1_validation::validate_input(&path) {
    Ok(src) => src,           // Extract success value, continue
    Err(err) => return PipelineResult { // Return error result immediately
        is_error: true,
        status_message: format!("■ Validation Error: {}", err),
        original_c_code: String::new(),
        // ... all other fields set to empty/false defaults ...
    },
};

```

Pattern	Explanation
<code>`match result { Ok(v) => v, Err(e) => return ... }`</code>	The "early error return" pattern. Much like <code>`?`</code> but for a non- <code>`Result`</code> return type (this function returns <code>`PipelineResult`</code> , not <code>`Result`</code>).
<code>`crate::module1_validation::validate_input(...)`</code>	Full crate-absolute path to avoid needing <code>`use`</code> imports at the top of the function.
<code>`let d1 = crate::module3_semantic::analyze_translation_unit_semantics(...)`</code>	Modules 3 and 4 NEVER fail (return plain values, not <code>`Result`</code>). No <code>`match`</code> needed.

Converting IdiomStatus to GUI tuples:

```
let idioms_status_display = report.idioms_summary.iter()
    .map(|i| (i.idiom_number, i.name.to_string(), i.detected, i.resolution.clone()))
    .collect();
```

`IdiomStatus` has a `&'static str` field (`name`) and `String` fields. The GUI uses tuples (`usize, String, bool, String`). `.to_string()` converts `&'static str` $\rightarrow$ `String`, and `.clone()` copies the `String` fields.

Lines 292-314: The `eframe::App::update()` — The GUI Event Loop

```
impl eframe::App for C2SafeRustApp {
    fn update(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
        // Poll channel for results from background thread
        if let Some(ref rx) = self.transpilation_rx {
            if let Ok(res) = rx.try_recv() {
                self.original_c_code = res.original_c_code;
                self.generated_rust_code = res.generated_rust_code;
                // ... copy all fields from PipelineResult to app state ...
                self.is_transpiling = false;
                self.transpilation_rx = None; // Channel consumed
            }
        }

        if self.is_transpiling {
            ctx.request_repaint(); // Keep animating the spinner
        }
    }
}
```

Code	Explanation
<code>`impl eframe::App for C2SafeRustApp`</code>	Implements the required trait — <code>`update()`</code> is the callback called every frame.
<code>`if let Some(ref rx) = self.transpilation_rx`</code>	<code>`ref`</code> borrows the <code>`Receiver`</code> without moving it. Just <code>`Some(rx)`</code> would move it, but we need <code>`self.transpilation_rx`</code> to remain <code>`Some`</code> until the result arrives.

Code	Explanation
<code>`rx.try_recv()`</code>	NON-BLOCKING receive. Returns <code>`Ok(result)`</code> if data is available, or <code>`Err(TryRecvError::Empty)`</code> if the thread is still running. Used instead of <code>`rx.recv()`</code> (which would block the GUI).
<code>`if let Ok(res) = rx.try_recv()`</code>	Only process if data arrived this frame.
<code>`self.transpilation_rx = None`</code>	Drop the channel — transpilation is complete.
<code>`ctx.request_repaint()`</code>	Tell egui to call <code>`update()`</code> again immediately (even if no input). Needed to animate the spinner. Without this, egui might not repaint until mouse input.

Lines 320-447: The Top Header Panel

```
egui::TopBottomPanel::top("top_panel")
    .frame(Frame::none()
        .fill(Color32::from_rgb(24, 27, 38))
        .inner_margin(Margin::symmetric(16.0, 12.0))
        .stroke(Stroke::new(1.0_f32, Color32::from_rgb(42, 47, 66))))
    .show(ctx, |ui| {
        ui.horizontal(|ui| { ... }); // Header row
        ui.add_space(6.0);
        Frame::none().fill(status_bg)...show(ui, |ui| { /* Status banner */ });
    });
```

The Status Banner — Colour-Coded State Machine:

```
let status_bg = if self.is_transpiling {
    Color32::from_rgb(99, 102, 241) // Indigo = Processing
} else if self.is_success {
    Color32::from_rgb(16, 185, 129) // Emerald = Success
} else if self.is_error {
    Color32::from_rgb(239, 68, 68) // Red = Error
} else {
    Color32::from_rgb(38, 44, 64) // Dark Neutral = Idle
};
```

State	Colour	Appearance
<code>`is_transpiling`</code>	Indigo <code>`#6366F1`</code>	Blue spinner + "■ Transpiling..."
<code>`is_success`</code>	Emerald <code>`#10B981`</code>	Green + "■ Transpilation Succeeded!"
<code>`is_error`</code>	Red <code>`#EF4444`</code>	Red + "■ Validation Error: ..."
Idle	Dark neutral <code>`#262C40`</code>	Neutral + initial message

The Run Transpilation Button — Two States:

```
if self.is_transpiling {
    // Disabled button (greyed out / spinner text)
    let disabled_btn = egui::Button::new(RichText::new("■ Transpiling...").strong())
        .fill(Color32::from_rgb(79, 70, 229))
        .rounding(Rounding::same(6.0))
        .min_size(Vec2::new(160.0, 36.0));
    ui.add_enabled(false, disabled_btn); // ui.add_enabled(false, ...) disables click
} else {
    let btn = egui::Button::new(RichText::new("■ Run Transpilation").strong())
        .fill(Color32::from_rgb(99, 102, 241))
        .rounding(Rounding::same(6.0))
        .min_size(Vec2::new(160.0, 36.0));
    if ui.add(btn).clicked() {
        self.start_transpilation();
    }
}
```

Code	Explanation
<code>`egui::Button::new(text)`</code>	Creates a button widget (not yet rendered).
<code>`.fill(Color32::from_rgb(r, g, b))`</code>	Background fill colour.
<code>`.rounding(Rounding::same(6.0))`</code>	Rounded corners with radius 6 pixels.
<code>`.min_size(Vec2::new(160.0, 36.0))`</code>	Minimum width 160px, height 36px.
<code>`ui.add_enabled(false, btn)`</code>	Add the button but disable clicking. <code>`false`</code> = disabled.
<code>`ui.add(btn).clicked()`</code>	Render the button AND check if it was clicked this frame.

File path input with Enter key:

```
let text_response = ui.add(
    egui::TextEdit::singleline(&mut self.file_path_input)
        .desired_width(240.0)
        .font(egui::TextStyle::Monospace),
);
if text_response.lost_focus() && ui.input(|i| i.key_pressed(egui::Key::Enter)) {
    self.load_file_from_path();
}
```

	Explanation		
<code>ui.add(...)</code>	A one-line text input bound directly to <code>file_path_input</code> . Edits update the String automatically.		
	Returns <code>`true`</code> if the text box just LOST keyboard focus (user pressed Tab or clicked away).		
	<code>i</code>	<code>i.key_pressed(egui::Key::Enter)`</code>	Closure-based
	Only reload if BOTH: focus was lost AND Enter was the cause.		

Lines 449-467: The KPI Metrics Bar

```
egui::TopBottomPanel::top("kpi_panel")...
    .show(ctx, |ui| {
        ui.horizontal(|ui| {
            render_kpi_card(ui, "■ C File",
                &format!("{}", self.c_lines),
                &format!("{}", self.c_bytes));
            ui.add_space(12.0);
            render_kpi_card(ui, "■■ AST Functions",
                &format!("{}", self.func_count),
                "Translation Unit");
            ui.add_space(12.0);
            render_kpi_card(ui, "■■ Call Graph",
                &format!("{}", self.call_nodes),
                &format!("{}", self.call_edges));
            ui.add_space(12.0);
            render_kpi_card(ui, "■ LLM Repair",
                &format!("{}", self.repair_iterations),
                if self.is_success { "Verified" } else { "Idle" });
        });
    });
```

`render_kpi_card(ui, title, main_val, sub_val)` draws a small card with three lines of text.

Lines 469-498: The Resizable Bottom Drawer with Three Tabs

```
egui::TopBottomPanel::bottom("bottom_panel")
    .resizable(true)
    .default_height(280.0)
    .height_range(120.0..=650.0)
    ...
    .show(ctx, |ui| {
        ui.horizontal(|ui| {
            ui.selectable_value(&mut self.active_tab, 0, "■ 6 C Idioms Taxonomy");
            ui.selectable_value(&mut self.active_tab, 1, "■ Executive Verification Report");
            ui.selectable_value(&mut self.active_tab, 2, "■■ Repair History Trace");
        });
        ui.separator();

        egui::ScrollArea::vertical().show(ui, |ui| {
            match self.active_tab {
                0 => render_idioms_tab(ui, &self.idioms_status_display),
                1 => render_executive_report_tab(ui, self),
                _ => render_repair_logs_tab(ui, &self.repair_logs),
            }
        });
    });
```

Code	Explanation
<code>.resizable(true)</code>	User can drag the panel's border to resize it.

Code	Explanation
<code>`.height_range(120.0..=650.0)`</code>	Resizable within this range only. <code>`...=`</code> is an inclusive range.
<code>`ui.selectable_value(&mut; self.active_tab, 0, "...")`</code>	A clickable tab label. When clicked, sets <code>`self.active_tab = 0`</code> . Auto-highlights when <code>`active_tab == 0`</code> .
<code>`egui::ScrollArea::vertical()`</code>	Adds a scrollbar to the content inside.
<code>`match self.active_tab { 0 => ..., 1 => ..., _ => ... }`</code>	Tab routing: render the correct tab based on <code>`active_tab`</code> . <code>`_`</code> is the wildcard — matches anything not already matched (here: <code>`2`</code> and anything else).

Lines 500-541: The Side-by-Side Code Panels

```
// Left: C Source (resizable)
egui::SidePanel::left("left_c_code_panel")
    .resizable(true)
    .default_width(ctx.screen_rect().width() * 0.48) // Start at 48% width
    .min_width(240.0)
    ...
    .show(ctx, |ui| {
        egui::ScrollArea::both().id_source("c_code_scroll").show(ui, |ui| {
            render_syntax_highlighted_code(ui, &self.original_c_code, false);
        });
    });

// Right: Rust Code (takes remaining space)
egui::CentralPanel::default()...
    .show(ctx, |ui| {
        egui::ScrollArea::both().id_source("rust_code_scroll").show(ui, |ui| {
            render_syntax_highlighted_code(ui, &self.generated_rust_code, true);
        });
    });
```

Code	Explanation
<code>`ctx.screen_rect().width() * 0.48`</code>	Dynamic default width: 48% of the screen. Recalculated on resize.
<code>`egui::ScrollArea::both()`</code>	Scrollable both horizontally and vertically. Important for long/wide code.
<code>`.id_source("c_code_scroll")`</code>	Unique ID for each scroll area — egui uses IDs to remember scroll positions. Without unique IDs, both panels would share state.
<code>`egui::CentralPanel::default()`</code>	Takes ALL remaining space after other panels claim their space.

Lines 705-793: The Syntax Highlighter

```
fn render_syntax_highlighted_code(ui: &mut egui::Ui, code: &str, is_rust: bool) {
    for (line_idx, line) in code.lines().enumerate() {
        ui.horizontal(|ui| {
            // Line number gutter
            ui.label(RichText::new(format!("{:3} ■ ", line_idx + 1))
                .color(Color32::from_rgb(107, 114, 128))
                .font(egui::FontId::monospace(13.0)));
            // Syntax-highlighted code tokens
            render_tokens_for_line(ui, line, is_rust);
        });
    }
}
```

`{:3}` in format string: right-align with width 3 — so `1` → " 1", `42` → " 42", `100` → "100".

This gives a proper line number gutter.

The Tokenizer `tokenize_code_line()` (Lines 742-761):

```
fn tokenize_code_line(line: &str) -> Vec<String> {
    let mut tokens = Vec::new();
    let mut current = String::new();

    for c in line.chars() {
        if c.is_alphanumeric() || c == '_' {
            current.push(c); // Build up word/identifier
        } else {
            if !current.is_empty() {
                tokens.push(current.clone());
                current.clear(); // Finish word, start fresh
            }
            tokens.push(c.to_string()); // Punctuation as its own token
        }
    }
    if !current.is_empty() { tokens.push(current); }
    tokens
}
```

For `"int x = 10;"`, this produces: `["int", " ", "x", " ", "=", " ", "10", ";"]`

Token Colour Mapping `get_token_color()` (Lines 763-792):

Category	Colour (<code>Color32`</code>)	Examples
Keywords	Purple <code>#C586C0`</code>	<code>`int`, `pub`, `fn`, `let`, `mut`, `struct`, `enum`, `return`</code>
Type names	Teal <code>#4EC9B0`</code>	<code>`i32`, `u8`, `f32`, `String`, `Vec`, `Option`</code>
Function names	Yellow <code>#DCDCAA`</code>	<code>`main`, `swap`, `malloc`, `free`, `println`</code>
Numbers	Orange-brown <code>#CE9178`</code>	Any token of all digits
Operators	Pink-red <code>#F43F5E`</code>	<code>`*`, `&`, `->`, `**`</code>

Category	Colour (`Color32`)	Examples
Default	Light grey `#D4D4D4`	Identifiers, punctuation

The Complete Module 7 Architecture

```

GUI starts (main.rs calls launch_gui())
    |
    v
C2SafeRustApp::new(cc)
-> set dark theme
-> load sample_test.c preview
    |
    [eframe event loop]
    |
    update() called ~60fps
    |
    |--- [User types path + clicks Load]
    |     load_file_from_path()
    |     -> show C source in left panel
    |
    |--- [User clicks ■ Run Transpilation]
    |     start_transpilation()
    |     -> channel(tx, rx) created
    |     -> thread::spawn(execute_transpilation_pipeline)
    |           -> Module 1 validate_input
    |           -> Module 2 preprocess_and_parse
    |           -> Module 3 analyze_translation_unit_semantics
    |           -> Module 4 run_interprocedural_analysis
    |           -> Module 5 generate_rust_code (writes output.rs)
    |           -> Module 6 run_repair_loop (up to 3 iterations)
    |           -> Module 7 assemble_translation_report
    |           -> tx.send(PipelineResult)
    |
    |--- [Frames during transpilation]
    |     rx.try_recv() returns Err (still running)
    |     ctx.request_repaint() keeps spinner going
    |
    |--- [Frame when pipeline finishes]
    |     rx.try_recv() returns Ok(result)
    |     -> Copy all PipelineResult fields to app state
    |     -> Left panel: original C code
    |     -> Right panel: generated Rust code
    |     -> Bottom: Idioms tab, Report tab, Repair History tab
    |     -> Status banner turns green ■ or shows iterations

```

Key Rust/egui Concepts Introduced in This Module

	Example	Meaning		
	<code>`name: &'static str`</code>	String slice living for the entire program — string literals.		
	On <code>`C2SafeRustApp`</code>	Auto-generate <code>`Default`</code> impl (all fields at zero/empty/false).		
<code>channel()`</code>	<code>`let (tx, rx) = channel()`</code>	Create a multi-producer single-consumer channel for thread communication.		
	<code>`Option>`</code>	The receiving end of an mpsc channel.		
	Non-blocking channel poll	Returns <code>`Ok(val)`</code> or <code>`Err(Empty)`</code> — never blocks.		
<code>move \</code>	<code>\</code>	<code>...)`</code>	Background thread	<code>`move`</code> takes variables.
<code>`</code>	Trait method	Called every frame — redraws the entire GUI.		
<code>`</code>	Force redraw	Without this, egui only repaints on user input events.		
<code>opt`</code>	Borrow inside Option	<code>`ref`</code> borrows without moving — <code>`opt`</code> remains <code>`Some`</code> .		
<code>mut; val,</code>	Tab button	Sets <code>`val = target`</code> when clicked; highlights when equal.		
<code>ch()`</code>	Both-axis scroll	Horizontal + vertical scrolling for wide/tall content.		
	Unique scroll ID	Prevents two scroll areas from sharing state.		
	Right-align width 3	<code>`1`</code> → <code>`" 1"`</code> , for line number gutters.		
<code>widget)`</code>	Disabled widget	Shows widget but click is ignored.		
	Struct update syntax	Fill remaining fields from the <code>`Default`</code> implementation.		
	Unused parameter prefix	Suppresses unused variable warning for intentionally unused parameter.		

Summary

Module 7 is the **presentation and coordination layer** — the final mile where analysis becomes visible:

`mod.rs` — Calls `eframe::run_native` with `NativeOptions` specifying a 1440×900 initial window. Uses the `ViewportBuilder` pattern to configure window title and size constraints. The app factory closure heap-allocates `C2SafeRustApp` for eframe.

`report_assembler.rs` — Takes outputs from all modules and builds the `TranslationReport`. Detects all 6 C idioms by querying D2 `InterproceduralSummaries`. Generates a formatted markdown string showing metrics, idioms table, and repair history. Uses `&'static str` for compile-time-fixed text.

`gui_app.rs` — The full egui desktop application:

- Custom dark navy theme with indigo accents
- File loading (`rfd` native dialog, path text input with Enter key)
- Background transpilation using `mpsc::channel` + `std::thread::spawn`
- Non-blocking result polling with `rx.try_recv()` + `ctx.request_repaint()` spinner
- 5 layout panels: top header, KPI bar, bottom drawer, left C panel, right Rust panel
- 3-tab bottom drawer: Idioms Grid | Executive Report | Repair History Logs
- Syntax highlighter with tokenizer + keyword/type/function/number colour mapping

Document generated for C2SafeRust-LLM Mini Project S3

Module 7: GUI & Report Assembler — Complete Line-by-Line Explanation